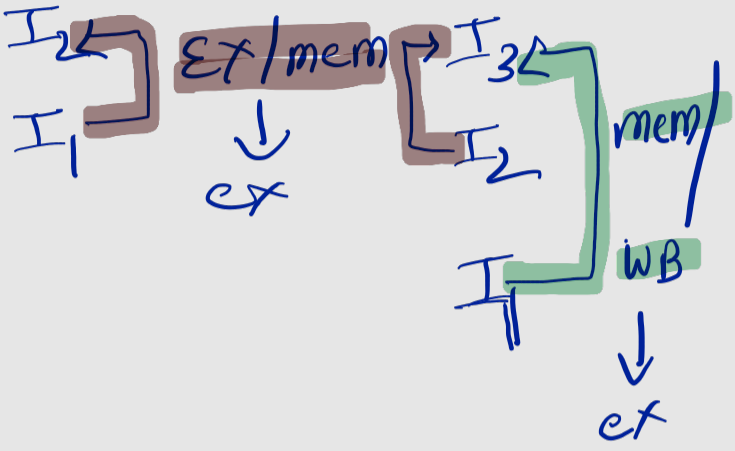
PART-A

(1)

	1	2	3	4	5
IF	I_1	I_2	I_3		
ID		I_1	I_2	I_3	
EX			I_1	I_2	
mem				I_1	
WB					I_1



I_1 → add s_0, t_0, t_1

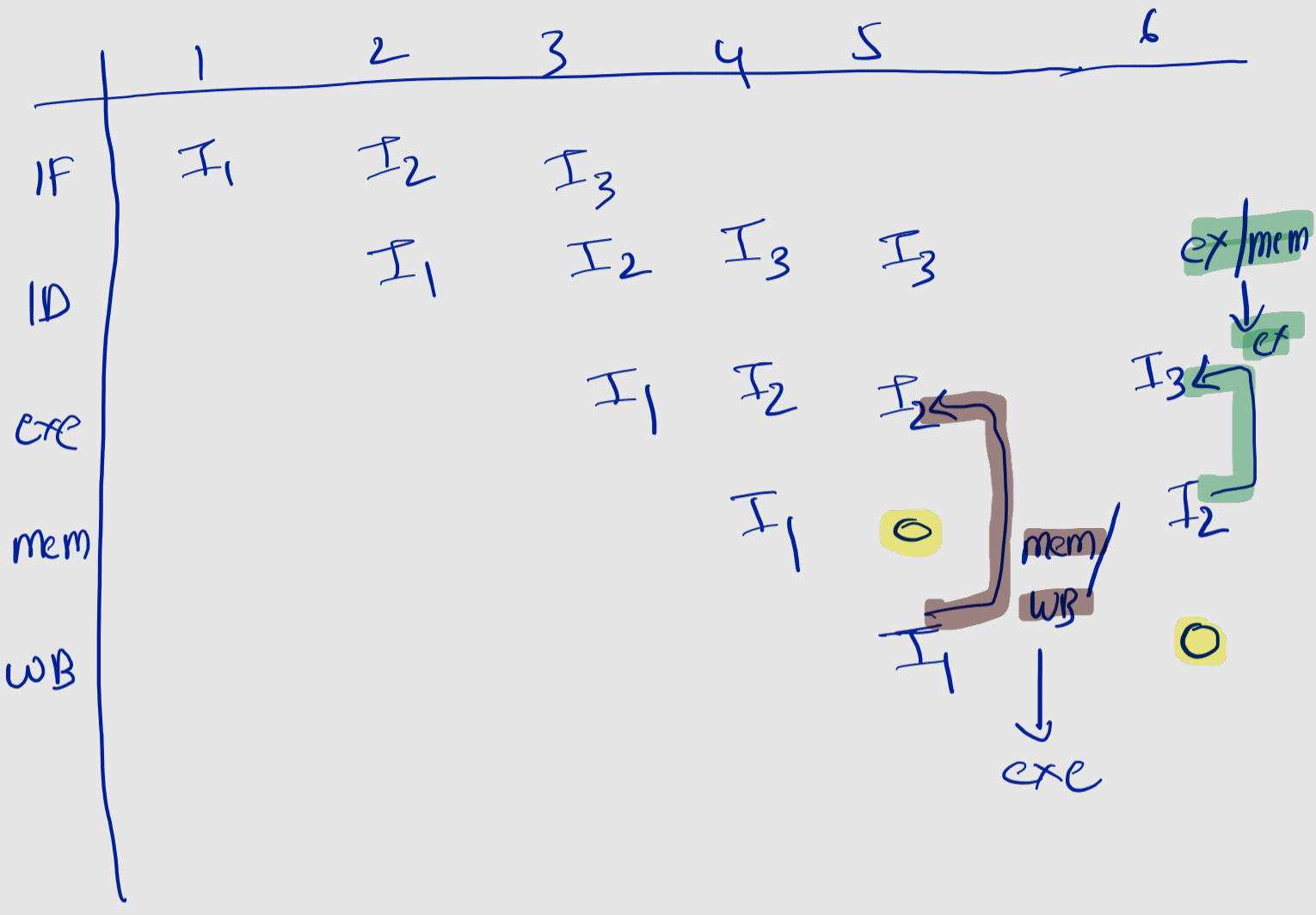
I_2 → sub s_1, s_0, t_2

I_3 → and s_2, s_1, s_0

(2)

Why forwading alone cannot save 1-cycle stall because the data from load availble after completion of memory stage so, the instruction in exe stage should stall 1-cycle to get the data.

(assuming memory access is 1-cycle)



$I_1 \rightarrow$ lw s_0, M[t_0]

$I_2 \rightarrow$ add s_1, s_0, t_1

$I_3 \rightarrow$ sub s_2, s_1, t_2

(3)

	1	2	3		
IF	I_1	I_2	I_4		
ID		I_1	I_2 (invalidated)	I_4	
exe			I_1	I_2 flushed	I_1
mem					
WB					

$I_1 \rightarrow$ beq to, t_1, target

$I_2 \rightarrow$ add s_0, s_1, s_2

$I_3 \rightarrow$ sub s_3, s_4, s_5

$I_4 \rightarrow$ or s_6, s_7, to (target)

only I_2 will be flushed

Part B

① Structural Hazard :- A structral hazard is a hazard which refers to a situation where There is resource conflict

ex:- in unified memory → load memory access and fetch.

Simultaneously access memory ←

Data Hazard :- when an instruction depends on the result of a previous instruction that is still moving through the pipeline and has not yet written it's result Back

ex:- add t_0, s_1, s_2

sub t_3, t_0, t_4

Control Hazards

when the pipeline cannot determine the next instruction to fetch because the decision (branch taken or not taken) depends on the outcome of a preceding branch instruction.

ex:

```
beq  t1, t2, target
add  s0, s1, s2
sub  s5, s4, s6
Target:
And  s7, s8, s9
```

(2) CPI for N Independent adds is "1"

CPI for N load instructions

$$= \frac{2N}{N} = 2$$

(3) cpu Time = IC × CPI × T_{cycle}.

To exceed IPC = 1 we'd need to execute more number of instructions To do that we need more than one datapath this not possible in single cycle.

(4) Comparitor should be added in Id stage.

2 cycles (ex) ⟶ 1 cycle (ID)

on correct prediction ⟶ 0 cycles

(5) RAW — no stall

add S_0 S_1 S_2

sub S_3 S_0 S_2

And S_4 S_3 S_2

RAW

lw s_0, M[to]

lw s_1, M[s_0]

(6) in - in order pipeline

- updates happens at WB stage.
- reads happen at ID
- memory operations happens at mem stage

∴ writes happen sequentially so, WAW is invisible

in in order pipelin with forwarding WAR is invisible.